- [x] Learn
- [x] Practice
- [x] Build Projects
- [x] Grow

Python Introduction

A simple language for big ideas

Simple
Powerful
Versatile

Satyarthi

1 What is Python?

Python is a high-level, interpreted, general-purpose programming language. It is known for its simple syntax and readability, which makes it easy to learn and use. Python can be used for web development, data analysis, artificial intelligence, automation and many other applications.

Why Learn Python?

- Easy to read and write
- Beginner friendly
- Large community support
- Used in real-world applications
- Works on different platforms (Windows, macOS, Linux)
- Great for career opportunities

2 First Python Program

Let's write a simple program to print a message.

Code :

```
1
2  print("Hello, World!")
```

Output

```
Hello, World!
```

This is the output printed on the screen.

Key Features

- [x] Simple and clean syntax
- [x] Interpreted language
- [x] Dynamically typed
- [x] Supports multiple paradigms (procedural, object-oriented, etc.)
- [x] Rich standard library
- [x] Widely used in industry and academia

3 Running a Python Program

You can run a Python program in different ways:

1. Using an IDE (like VS Code, PyCharm, etc.)
2. Using the command line / terminal
3. Using an online compiler (like Replit, Google Colab, etc.)

Did You Know?

Python was created by Guido van Rossum and first released in 1991.

4 Python in Real Life

Python is used in many popular applications and companies:

Instagram	Spotify	YouTube	NASA	Google
(Backend)	(Data Analysis)	(Backend)	(Research)	(Many tools)

Key Takeaways

- [ ] Python is easy to learn.
- [ ] It can be used for many applications.
- [ ] You can run Python code in multiple ways.
- [ ] It has a large community and great industry support.

"A journey of a thousand programs begins with a single print()." – Satyarthi

A variable is like a box that holds a value. ☺

Python Variables

Store data, use it, change it!

Learn
Practice
Code
Improve
Grow

Satyarthi

1 What is a Variable?

A variable in Python is a named memory location used to store a value. You can store different types of data like numbers, text, etc. and use them later in your program.

2 How to Create a Variable?

You can create a variable by simply writing the variable name, an equal sign (=) and the value.

Code:

```
name = "Satyarthi"
age = 19
height = 169.5
is_student = True
```

No need to declare the type in Python!

3 Valid Variable Names

A variable name can contain:

- Letters (a–z, A–Z)
- Digits (0–9)
- Underscore (_)
- It must start with a letter or underscore
- It cannot be a Python keyword
- Variable names are case-sensitive (name and Name are different).

4 Examples of Valid Names

- ✓ name
- ✓ _age
- ✓ student1
- ✓ total_marks
- ✓ price2026
- ✓ myValue

5 Invalid Names

- ✗ 1name (cannot start with digit)
- ✗ total-marks (hyphen not allowed)
- ✗ class (keyword)
- ✗ name@ (special character)
- ✗ for (keyword)

6 Different Types of Values

A variable can store different types of data.

Data Type	Example	Variable
String (text)	"Hello"	name
Integer (whole number)	25	age
Float (decimal number)	169.5	height
Boolean (True/False)	True	is_student

7 Using Variables

You can use variables in your program anytime after assigning a value.

Code:

```
name = "Satyarthi"
age = 19
print(name)    # Output: Satyarthi
print(age)     # Output: 19
```

8 Changing the Value

You can change the value of a variable at any time. The new value will replace the old value.

Code:

```
age = 19
print(age)    # 19
age = 20
print(age)    # 20
```

The old value is lost once a new value is assigned.

9 Key Takeaways

- ✓ A variable stores a value.
- ✓ You don't need to declare the type.
- ✓ Python automatically understands the type.
- ✓ You can store any type of data.
- ✓ Variable names are case-sensitive.
- ✓ You can change the value anytime.
- ✓ Use meaningful names.

"Small variables can store big dreams." – Satyarthi ☺

Practice
Think
Code
Improve

Python Variables

Small Variables, Big Possibilities

Learn
Practice
Build
Grow

Question 1

(Variables in Python)

Write a Python program to take the user's **name** and **age** as input and print a message using these variables.

Code :

```
1  name = input("Enter your name: ")
2  age = int(input("Enter your age: "))
3
4  print("Hello, " + name + "!")
5  print("You are", age, "years old.")
```

Sample Input :

```
Enter your name: Satyarthi
Enter your age: 19
```

Output :

```
Hello, Satyarthi!
You are 19 years old.
```

Explanation :

1. We use input() to take data from the user.
2. The input for age is converted to integer using int().
3. The values are stored in the variables name and age.
4. We use print() to display the message on the screen.
5. The output shows the user's name and age using the stored variables.

Key Takeaways :

- ✅ A variable is used to store data.
- ✅ We can take input from the user using input().
- ✅ We can use different data types (string, int, float, etc.).
- ✅ Meaningful variable names make the program easy to understand.
- ✅ Variables help us make interactive programs.

"Start with small programs, create big opportunities." – Satyarthi

Keep Practicing

Learn
Practice
Build Logic
Grow

Python Data Types

Small
Steps
Big
Programs

Satyarthi

Different types of data for different needs

1 What are Data Types?

Data types define the type of value a variable can store. They tell Python what kind of data it is and what operations can be performed on it.

2 Common Data Types in Python

Python has several built-in data types. The most commonly used are:

Data Type	Description	Example
int	Whole numbers	10, -5, 100
float	Decimal numbers	3.14, -0.5
str	Text (string)	"Hello", 'Python'
bool	Boolean values	True, False
list	Ordered collection (mutable)	[1, 2, 3]
tuple	Ordered collection (immutable)	(1, 2, 3)
set	Unordered collection (unique)	{1, 2, 3}
dict	Key-value pairs	{"name": "Satyarthi"}

4 Checking the Data Type

We can use the type() function to check the data type of a variable.

Code :

```
1  x = 10
2  print(type(x))
3  name = "Satyarthi"
3  print(type(name))
```

Output :

```
<class 'int'>
<class 'str'>
```

5 Type Conversion

We can convert one data type to another using built-in functions.

Code :

```
1  num_str = "25"
2  num_int = int(num_str)
3  print(num_int)
4  print(type(num_int))
```

Output :

```
25
<class 'int'>
```

Why are Data Types Important?

- Help Python understand the data
- Decide what operations are allowed
- Make the program more efficient
- Prevent errors
- Used in all real-world applications

3 Examples of Each Data Type

Code :

```
1  x = 10                    # int
2  y = 3.14                  # float
3  name = "Satyarthi"        # str
4  is_student = True         # bool
5  fruits = ["apple", "banana"]   # list
6  numbers = (1, 2, 3)       # tuple
7  unique = {1, 2, 3}        # set
8  info = {"name": "Satyarthi", "age": 19} # dict
```

Output (Type of each variable) :

```
1  10 <class 'int'>
2  3.14 <class 'float'>
3  Satyarthi <class 'str'>
4  True <class 'bool'>
5  ['apple', 'banana'] <class 'list'>
6  (1, 2, 3) <class 'tuple'>
7  {1, 2, 3} <class 'set'>
8  {'name': 'Satyarthi', 'age': 19} <class 'dict'>
```

6 Key Takeaways

- ☑ Data types define the kind of value stored in a variable.
- ☑ Python has int, float, str, bool, list, tuple, set, dict.
- ☑ Use type() to check the data type.
- ☑ You can convert between data types.
- ☑ Choosing the right data type helps write better programs.

Did You Know?

Python automatically detects the data type. You do not need to declare it explicitly!

"Different data types, endless possibilities." – Satyarthi

Learn
Practice
Build Logic
Improve ☺

Python Data Types

Understand Different Types of Data

Small
Steps
Big
Programs ☺

Satyarthi

Question 1

(Data Types in Python)

Write a Python program to take several values of different data types from the user (integer, float, string and boolean) and print their values along with their data types using the type() function.

Code :

```
1  num = int(input("Enter an integer: "))
2  decimal = float(input("Enter a float: "))
3  text = input("Enter a string: ")
4  flag = input("Enter a boolean (True/False): ")
5  flag = flag == "True"  # convert to boolean
6
7  print("Integer:", num, "- Type:", type(num))
8  print("Float:", decimal, "- Type:", type(decimal))
9  print("String:", text, "- Type:", type(text))
10 print("Boolean:", flag, "- Type:", type(flag))
```

Sample Input :

```
Enter an integer: 25
Enter a float: 3.14
Enter a string: Python
Enter a boolean (True/False): True
```

Output :

```
Integer: 25 - Type: <class 'int'>
Float: 3.14 - Type: <class 'float'>
String: Python - Type: <class 'str'>
Boolean: True - Type: <class 'bool'>
```

Explanation :

1. We take input from the user for different data types.
2. `int()` converts the input to an integer.
3. `float()` converts the input to a float.
4. `input()` takes a string.
5. For boolean, we convert the input string to a boolean value using (flag == "True").
6. We use `type()` to find and display the data type of each variable.

Key Takeaways :

- ☑ Python has different built-in data types (int, float, str, bool, etc.).
- ☑ Use `type()` to check the data type of a variable.
- ☑ You can convert user input to a specific data type using int(), float(), etc.
- ☑ Understanding data types helps in writing better and error-free programs.

"Same data, different types, different possibilities."

– Satyarthi ☺

Code
Practice
Repeat ☺

- [x] Learn
- [x] Practice
- [x] Build Logic
- [x] Improve

Python Data Types

Same data, different types, different possibilities

Small Steps Big Programs

Satyarthi

1 Question

Write a Python program to take input from the user as an integer, float, string and boolean, and print their values along with their data types using the type() function.

One Question Many Concepts

Code :

```
1  num = int(input("Enter an integer: "))
2  dec = float(input("Enter a float: "))
3  text = input("Enter a string: ")
4  flag = input("Enter a boolean (True/False): ")
5  flag = flag == "True"  # convert to boolean
6
7  print("Integer:", num, "- Type:", type(num))
8  print("Float:", dec, "- Type:", type(dec))
9  print("String:", text, "- Type:", type(text))
10 print("Boolean:", flag, "- Type:", type(flag))
```

Sample Input :

```
Enter an integer: 10
Enter a float: 3.5
Enter a string: Python
Enter a boolean (True/False): True
```

Output :

```
Integer: 10 - Type: <class 'int'>
Float: 3.5 - Type: <class 'float'>
String: Python - Type: <class 'str'>
Boolean: True - Type: <class 'bool'>
```

Explanation :

1. We take input from the user for different data types.
2. `int()` converts the input to an integer.
3. `float()` converts the input to a float.
4. `input()` takes a string.
5. For boolean, we convert the input string to a boolean using (flag == "True").
6. We use `type()` to find and display the data type of each variable.

Common Data Types in Python

Data Type	Example
int	10, -5, 100
float	3.14, -0.5
str	"Hello", "Python"
bool	True, False
list	[1, 2, 3]
tuple	(1, 2, 3)
set	{1, 2, 3}
dict	{"name": "Satyarthi"}

Key Takeaways :

- [x] Python has different built-in data types (int, float, str, bool, etc.).
- [x] Use `type()` to check the data type of a variable.
- [x] You can convert user input to a specific data type.
- [x] Different data types are useful for different kinds of data.
- [x] Understanding data types helps in writing better and error-free programs.

Did You Know?

In Python, you don't need to declare the data type explicitly. Python automatically detects the data type!

"Different data types, endless possibilities." – Satyarthi

Keep Learning Keep Growing

- Learn
- Practice
- Build Logic
- Improve

Python Type Casting

Convert data from one type to another

Small Steps Big Programs

Satyarthi

1 What is Type Casting?

Type casting is the process of converting a value from one data type to another in Python. It helps us to use the data in the required format for performing operations.

Why is Type Casting Important?

- Helps in performing operations between different data types
- Converts user input (which is always a string) to other types
- Prevents type errors
- Useful in real-world applications
- Makes the program more flexible

2 Types of Type Casting

Python provides two types of type casting:

1. **Implicit Type Casting**
   (Automatic conversion by Python)
2. **Explicit Type Casting**
   (Manual conversion using built-in functions)

3 Example Program

Let's take input from the user and convert it to different data types.

Code :

```
1  num = int(input("Enter a number: "))
2  dec = float(input("Enter a decimal: "))
3  text = str(input("Enter a string: "))
4  print("Integer:", num, "- Type:", type(num))
5  print("Float:", dec, "- Type:", type(dec))
6  print("String:", text, "- Type:", type(text))
```

4 Sample Input & Output

Sample Input :

```
Enter a number: 25
Enter a decimal: 3.14
Enter a string: Python
```

Output :

```
Integer: 25 - Type: <class 'int'>
Float: 3.14 - Type: <class 'float'>
String: Python - Type: <class 'str'>
```

5 Common Type Casting Functions

Function	Description
int()	Converts to integer
float()	Converts to float (decimal)
str()	Converts to string
bool()	Converts to boolean
list()	Converts to list
tuple()	Converts to tuple
set()	Converts to set
dict()	Converts to dictionary

6 Key Takeaways

- [x] Type casting converts data from one type to another.
- [x] Use int(), float(), str(), etc. for explicit casting.
- [x] Python also performs implicit casting automatically.
- [x] Type casting is useful for taking user input and operations.
- [x] It helps in writing flexible and error-free programs.

Did You Know?

The input() function in Python always takes input as a string. So, we often use type casting to convert it to int, float, etc.

"Right type, right value, brighter code."

– Satyarthi

Keep Learning Keep Growing

- [x] Learn
- [x] Practice
- [x] Build Logic
- [x] Grow

Python

Type Casting

Convert data from one type to another

- [x] Small
- [x] Steps
- [x] Big
- [x] Progress

Satyarthi

Question 1

(Type Casting in Python)

Write a Python program to take a number as input from the user (as a string) and convert it to an integer, a float and a string again using type casting. Print the value and its type at each step.

Code :

```
num_str = input("Enter a number: ")
num_int = int(num_str)
num_float = float(num_str)
num_again = str(num_int)

print("As Integer:", num_int, "- Type:", type(num_int))
print("As Float  :", num_float, "- Type:", type(num_float))
print("As String :", num_again, "- Type:", type(num_again))

```

Sample Input :

Enter a number: 25

Output :

As Integer: 25 - Type: <class 'int'>
As Float : 25.0 - Type: <class 'float'>
As String : 25 - Type: <class 'str'>

Explanation :

1. The input() function always takes input as a string.
2. We use int() to convert the string to an integer.
3. We use float() to convert the string to a float (decimal number).
4. We use str() to convert the integer back to a string.
5. The type() function shows the data type of each variable.

Key Takeaways :

- [x] Type casting helps convert data from one type to another.
- [x] Python provides int(), float(), str() and other functions.
- [x] Input from the user is always a string.
- [x] Use type() to check the data type of a variable.
- [x] Type casting is useful in real-world programs.

Did You Know?

You can also convert between other types like int(), float(), str(), list(), tuple(), etc. using type casting!

"Same data, different types, different possibilities."

– Satyarthi

Code
Practice
Repeat

- [x] Learn
- [x] Practice
- [x] Build Projects
- [x] Grow

Python Operators

Perform operations on values and variables

Small Steps Big Programs

Satyarthi

1 What are Operators?

Operators are special symbols in Python that perform operations on values and variables. They are used to carry out tasks like arithmetic calculations, comparisons, logical checks, and more.

Why Use Operators?

- Help in performing calculations
- Compare values
- Make decisions in programs
- Work with conditions (True/False)
- Used in loops and other control structures
- Essential for real-world programming
- Make the code more powerful and dynamic

2 Types of Operators in Python

Python provides different types of operators:

1. Arithmetic Operators (+, -, *, /, % ...)
2. Relational (Comparision) Operators (==, !=, >, < ...)
3. Logical Operators (and, or, not)
4. Assignment Operators (=, +=, -= ...)
5. Bitwise Operators (&, |, ^, ~, <<, >>)
6. Membership Operators (in, not in)
7. Identity Operators (is, is not)

3 Arithmetic Operators

Used to perform mathematical operations.

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 2	2.5
%	Modulus (Remainder)	5 % 2	1
**	Exponentiation	5 ** 2	25
//	Floor Division	5 // 2	2

4 Comparison Operators

Used to compare two values. It returns True or False.

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater than or equal to	5 >= 5	True
<=	Less than or equal to	5 <= 2	False

5 Logical Operators

Used to combine conditional statements.

Operator	Meaning	Example	Result
and	True if both are True	True and False	False
or	True if at least one is True	True or False	True
not	Reverse the result	not True	False

6 Assignment Operators

Used to assign values to variables.

Operator	Meaning	Example	Same As
=	Assign	x = 5	x = 5
+=	Add and assign	x += 3	x = x + 3
-=	Subtract and assign	x -= 2	x = x - 2
*=	Multiply and assign	x *= 4	x = x * 4
/=	Divide and assign	x /= 2	x = x / 2
%=	Modulus and assign	x %= 3	x = x % 3

7 Other Important Operators

Membership Operators

Operator	Meaning	Example
in	True if value exists	5 in [1, 2, 5] True
not in	True if value does not exist	3 not in [1, 2, 5] True

Identity Operators

Operator	Meaning	Example
is	True if both refer to same object	a is b
is not	True if not the same object	a is not b

Key Takeaways

- [x] Operators help us perform operations on data.
- [x] Python provides different types of operators.
- [x] They are widely used in conditions, loops and real-world programs.
- [x] Understanding operators is essential for writing efficient code.

Did You Know?

You can use multiple operators in one expression like: x = 5 + 3 * 2

(Python follows BODMAS / precedence rules!)

"Operators turn simple values into powerful possibilities." – Satyarthi

Keep Learning Keep Growing

- [x] Learn
- [x] Practice
- [x] Solve Problems
- [x] Grow

Python

Practice Question

Learn • Practice • Improve

Simple
Powerful
Versatile

Satyarthi

1 Question

Take a number as input from the user and check whether it is even or odd.

Example Input / Output

Input	Output
4	Even number
7	Odd number
0	Even number
-3	Odd number

2 Python Program

Code :

```
1  num = int(input("Enter a number: "))
2  if num % 2 == 0:
3      print("Even number")
4  else:
5      print("Odd number")
```

3 Sample Run

```
Enter a number: 4
Even number
```

```
Enter a number: 7
Odd number
```

4 How It Works?

1. The user enters a number.
2. The number is converted to an integer using `int()`.
3. The `%` operator gives the remainder when divided by 2.
4. If the remainder is 0, the number is even, otherwise it is odd.

5 Key Takeaways

- [x] Use `input()` to take user input.
- [x] Use `int()` to convert input to integer.
- [x] `%` is the modulus operator (gives remainder).
- [x] A number is even if num % 2 == 0.
- [x] Otherwise, it is odd.
- [x] This concept is widely used in real-world programs (validation, games, etc.).

Did You Know?

0 is an even number because `0 % 2 == 0`.

"Practice today, stronger tomorrow."

– Satyarthi

- [x] Learn
- [x] Practice
- [x] Solve Problems
- [x] Grow

Python

Practice Question

Small Steps • Big Progress

Simple
Powerful
Versatile

Satyarthi

1 Question

Take two numbers as input from the user and find the greater number. If both numbers are equal, print "Both numbers are equal".

2 Example Input / Output

Input	Output
10 20	20 is greater
35 12	35 is greater
7 7	Both numbers are equal
-5 -8	-5 is greater

3 Python Program

Code :

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
if a > b:
    print(a, "is greater")
elif b > a:
    print(b, "is greater")
else:
    print("Both numbers are equal")
```

4 Sample Run

```
Enter first number: 10
Enter second number: 20
20 is greater
```

```
Enter first number: 35
Enter second number: 12
35 is greater
```

```
Enter first number: 7
Enter second number: 7
Both numbers are equal
```

```
Enter first number: -5
Enter second number: -8
-5 is greater
```

5 How It Works?

1. Take two inputs from the user using `input()`.
2. Convert them to integers using `int()`.
3. Compare the numbers using `>` operator.
4. If the first number is greater, print it.
5. If the second number is greater, print it.
6. If both are equal, print a message.

6 Key Takeaways

- [x] Use `input()` to take user input.
- [x] Use `int()` to convert input to integer.
- [x] Use `if-elif-else` for decision making.
- [x] Comparison operators help us compare values.
- [x] This logic is useful in many real-world programs.

Did You Know?

You can also use the `max()` function to find the greater number!

Example: `max(a, b)`

"Practice coding,
build a better you."
– Satyarthi

Keep
Learning
Keep
Growing

- [x] Learn
- [x] Practice
- [x] Solve Problems
- [x] Grow

Python

Practice Question

Small Steps • Big Progress

Simple
Powerful
Versatile

Satyarthi

1 Question

Take two numbers as input from the user and find the greater number. If both numbers are equal, print "Both numbers are equal".

2 Example Input / Output

Input	Output
10 20	20 is greater
35 12	35 is greater
7 7	Both numbers are equal
-5 -8	-5 is greater

3 Python Program

Code :

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
if a > b:
    print(a, "is greater")
elif b > a:
    print(b, "is greater")
else:
    print("Both numbers are equal")
```

4 Sample Run

```
Enter first number: 10
Enter second number: 20
20 is greater
```

```
Enter first number: 35
Enter second number: 12
35 is greater
```

```
Enter first number: 7
Enter second number: 7
Both numbers are equal
```

```
Enter first number: -5
Enter second number: -8
-5 is greater
```

5 How It Works?

1. Take two inputs from the user using `input()`.
2. Convert them to integers using `int()`.
3. Compare the numbers using `>` operator.
4. If the first number is greater, print it.
5. If the second number is greater, print it.
6. If both are equal, print a message.

6 Key Takeaways

- [x] Use `input()` to take user input.
- [x] Use `int()` to convert input to integer.
- [x] Use `if-elif-else` for decision making.
- [x] Comparison operators help us compare values.
- [x] This logic is useful in many real-world programs.

Did You Know?

You can also use the `max()` function to find the greater number!

Example: `max(a, b)`

"Practice coding,
build a better you."
– Satyarthi

Keep
Learning
Keep
Growing